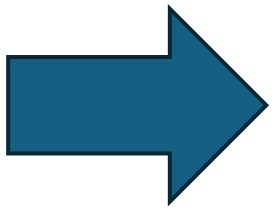
Converting C++ to Rust



10/8/2025

Dave Henderson

Introduction

- Why convert your C++ application to Rust?
  - Runsafe Security
  - Secure by Design
  - Avoid memory safety vulnerabilities
    - See "CISA Case for Memory Safer Roadmaps
  - In safe Rust there is no undefined behavior
  - There are Memory Safety Guarantees
  - No Garbage Collector
  - Avoid data races, common in C/C++, especially with multithreaded applications

Introduction

- There are several approaches to conversion from C++ to Rust
  - Total rewrite in Rust from scratch ("Big Bang" approach)
    - Potentially a long time before you have functional, testable code
    - Second System effect
  - Automatic conversion
    - Doesn’t produce idiomatic, safe Rust code
    - May be good way to get started quickly, refactoring into idiomatic Rust as you go
    - Generating FFI bindings is a halfway approach to introducing Rust to C/C++ and vice versa
  - Gradual introduction of Rust by rewriting modules or functionality in a piece-wise fashion
    - Always have a working version of the codebase
    - Critical sections can be handled first
    - Make use of built-in test framework of Rust
    - Use automatic conversion to make progress, refactor as needed
    - Avoids/postpones refactoring/rewrite so original and new source can be compared
    - Recommend this approach for both schedule and cost reasons
    - End point can be Rust code only or a hybrid of C/C++ and Rust, pragmatism applies here
  - Conversion using AI
    - Not covered in this presentation, but there has been rapid progress in this area

Gradual Introduction of Rust into C++

- First, understand the C/C++ code of the application you are converting
- Review algorithms, data structures in your C/C++ code
- For embedded systems especially, review platform-specific items in your C/C++ code including:
  - Inline assembler code
  - C/C++ specific drivers, low level hardware interfaces
  - Direct access to memory and devices

Gradual Introduction of Rust into C++

- Identify the critical modules or features that should be converted first, including:
  - Security critical functionality
  - Foundational modules, libraries, frameworks that determine the structure of your application
  - Drivers and interfaces that may need heavy modification to work with Rust
- Best practice when converting is incremental rewriting along clear boundaries
- Always be able to regression test, unit test
  - Add tests as Rust modules are added
  - Be able to regression test the application at all times
  - Make regression testing part of your CI/CD workflow

Gradual Introduction of Rust into C++

- Testing:
  - Compare byte for byte results from old to new
    - Use "Memory Access" trait to examine memory modifications
  - Add tests for any bugs found (your porting may reveal latent defects!)
- TPMs (Technical Performance Measures)
  - Add instrumentation to track memory, execution time, image size
  - Periodically measure and compare to original C++ metrics
  - Target optimization effort where needed, a focused effort
  - Set up profiling along with your automated regression testing

Porting Approach - Automation

- **C2Rust**
- Translates C99 Compliant C code to semantically equivalent Rust
- Provides:
  - A C to Rust translator
  - Rust code refactoring tool
  - Tools to cross-check execution of the C code against your new Rust code
- Pros:
  - Good starting point for converting and refactoring the code
- Cons:
  - Does C conversion, not a C++ converter
  - Doesn’t result in idiomatic Rust code
  - Produces unsafe Rust code

Porting Approach - Automation

bindgen

- Generates FFI (Foreign Function Interfaces) from C and C++ header files
- Permits a piecewise approach to converting C++ code
- Pros:
  - Permits a piecewise approach to converting C++ code
  - Good starting point for converting and refactoring the code
- Cons:
  - Does C conversion, not a C++ converter
  - Doesn’t result in idiomatic Rust code
  - Produces unsafe Rust code

- Reference: Bindgen

Porting Approach - Automation

cbindgen

- Produces C header files from Rust structures, functions and constants
- Allows you to call Rust code from existing C code

```rust
use libc::c_char;
use std::ffi::CStr;

#[no_mangle]
pub unsafe extern fn print_hello(name: *const c_char) {
    let name = CStr::from_ptr(name);
    let name = name.to_str().unwrap();
    println!("Hello {}!", name);
}
```

produces...

```rust
#include <cstdarg>
#include <cstdint>
#include <cstdlib>
#include <ostream>
#include <new>

extern "C" {

void print_hello(const char *name);

} // extern "C"
```

- Good for bringing in Rust code early in the conversion process
- Usage: `cbindgen --config cbindgen.toml --crate my_rust_library --output my_header.h` (Use `–lang c` to produce a header for C)
- Reference: https://github.com/mozilla/cbindgen/tree/master

Structure

C++ and Rust structure their programs differently

C++	Rust	
namespace #include	module	In Rust, just name the file and you have a module Or use pattern myapp/mod.rs to pull in subordinate files
Source (.c/.cpp) and header (.h/.hpp) files	.rs files	No need for header files .rs files can serve as modules
Cmake, make, etc.	cargo	

Language Features Compared

C++	Rust	
Compile flags, #pragma, #define, #ifdef/#else/#endif, inline, const, volatile	*attributes* notation e.g. #[test] or #![test] to denote a unit test	In Rust, attributes can be used for traits, linting rules, compiler feature selection, conditional compilation, etc.
Const_cast<T>, static_cast<T>, Reinterpret_cast<T>	...as... Into<Xyz>, From<Xyz> traits	Rust disallows casting in the C++ sense
std:vector, std::list, std::set, std::map	Std::vec::Vec, std::collections::LinkedList, std::collections::HashSet Std::collections::HashMap	
num, union	enum, union (but unsafe!)	Enum is terser and safer in Rust, union can be useful when interfacing with legacy C/C++ code
setjmp, longjmp, throw/catch, returning error code from function	panic!("Yikes"); Result<T, E> *Option* enum	`enum Result<T, E> { Ok(T), Err(E) }` `enum Option<T> { None Some(T) }`
? (ternary operator)	`if y / 2 == 4 { true } else { false };`	

Language Features Compared

C++	Rust	
switch	match	*match* is more flexible and works on integers, ranges of integers, bools, enums, structs, etc. Default handler always required
char *, w_char_t *, std::string, std::wstring	Char, str, std::String, OSString, OSStr	Use OSString and OSStr types for FFI to interface with C/C++ code Rust uses UTF-8 encoding for strings
class, struct	struct, impl	Use traits in Rust for inheritance In Rust, must initialize all members when creating struct, make use of Option<>
void	Unit	Type of Unit is () and it has only one value (), returned for blocks that evaluate to nothing

Pointers: char * = "Xyz";
NULL, nullptr
	Raw pointers:
	let offset: u16 = 55;
	let x: * u16 = &offset;
		Useful for accessing C API but in most cases unsafe

Further Resources

- Getting started with Rust
- Online documentation for Rust
- The Rustonomicon
- Getting started - Rust Programming Language
- Install Rust - Rust Programming Language
- "The Rust Programming Language" - https://rust-lang.github.io/rustup/installation/index.html
- C++ to Rust conversion guidance
- GitHub - locka99/cpp-to-rust-book
- https://medium.com/@AlexanderObregon/porting-c-c-libraries-to-rust-a-practical-guide-95e341bd1737

Questions?